

Piškvorky

KIV/UPS – Semestrální práce

student: Ondřej Patejdl
osobní číslo: A23B0413P
email: opatejdl@students.zcu.cz
datum: 10.1.2026

Obsah

1	Zadání	2
1.1	Specifikace mé práce	2
2	Popis hry	2
3	Programátorská dokumentace	3
3.1	Popis protokolu	3
3.1.1	Formát zpráv	3
3.1.2	Zprávy protokolu	3
3.1.3	Detailní popis zpráv protokolu	4
3.2	Popis stavů klienta	7
3.2.1	Seznam stavů	7
3.2.2	Diagram stavů klienta	8
3.3	Implementace serveru	9
3.4	Implementace klienta	9
4	Uživatelská dokumentace	10
4.1	Obsluha serveru	10
4.1.1	Kompilace serveru	10
4.1.2	Spuštění serveru	10
4.2	Obsluha klienta	11
4.2.1	Příprava prostředí	11
4.2.2	Spuštění klienta	11
4.3	Základní ovládání klienta	12
4.3.1	Přihlášení do systému (LoginScene)	12
4.3.2	Hlavní lobby (LobbyScene)	12
4.3.3	Čekací místnost (WaitingScene)	13
4.3.4	Průběh herního zápasu (GameScene)	13
4.3.5	Výsledek hry a odvěta	14
5	Závěr	15

1 Zadání

Cílem semestrální práce z předmětu KIV/UPS je vytvoření síťové hry pro více hráčů založené na architektuře server-klient (1:N) určené pro platformu PC. Pro implementaci serverové části je stanoven jazyk C/C++. Klientská část musí být vyvinuta ve vysokoúrovňovém jazyce (např. Java, C#, Kotlin) s grafickým uživatelským rozhraním.

Mezi klíčové požadavky patří:

- **Komunikační protokol:** Vlastní textový protokol postavený nad transportní protokolem TCP nebo UDP bez využití šifrování.
- **Síťová komunikace:** Server musí využívat výhradně BSD sockety a klient nativní podporu standardní knihovny daného jazyka; použití knihoven třetích stran pro komunikaci či serializaci je zakázáno.
- **Stabilita a robustnost:** Aplikace musí ošetřovat všechny výjimky, předcházet pádům (např. segfault) a nesmí docházet k zablokování (deadlock).
- **Paralelizace:** Server musí být schopen obsluhovat více herních místností současně, aniž by se navzájem ovlivňovaly.
- **Chybové stavy:** Obě aplikace musí korektně reagovat na nevalidní zprávy a umožnit zotavení po výpadku spojení.

Kompletní specifikace požadavků je uvedena na stránce:
<https://home.zcu.cz/~ublm/files/PozadavkyUPS.pdf>

1.1 Specifikace mé práce

Cílem mé práce bylo implementovat hru Piškvorky 3x3 pomocí síťového protokolu TCP. Pro implementaci serveru byl využit nízkoúrovňový jazyk C++. Naopak pro klientskou stranu byl využit vysokoúrovňový jazyk python.

2 Popis hry

Piškvorky ve variantě 3x3 jsou logickou tahovou hrou určenou pro dva hráče. Celá hra probíhá ve čtvercovém poli o rozměrech 3x3, kde se soupeři pravidelně střídají v pokládání svých symbolů (nejčastěji křížků a koleček). Hlavním cílem je obsadit tři pole v nepřerušené horizontální, vertikální nebo diagonální linii. Pokud se jednomu z hráčů povede vytvořit zmíněnou linii stává se vítězem. Naopak pokud je pole zcela zaplněno symboly a žádnému z hráčů se nepodařilo vytvořit vítěznou řadu, končí hra remízou.

3 Programátorská dokumentace

3.1 Popis protokolu

3.1.1 Formát zpráv

Komunikace mezi serverem a klientem probíhá pomocí textového protokolu založeném na principu protokolu TCP. Každá zpráva odeslaná mezi klientem a serverem musí striktně dodržovat následující strukturu,

`<hlavička>|<identifikátor>|<parametr1>|...|<parametrn>\n`

kde:

- **Hlavička:** Jedná se o magickou konstantu, kterou musí začínat každá validní zpráva. Její hodnota je 0P23 a slouží jako jednoznačný identifikátor protokolu.
- **Oddělovač:** Slouží k oddělení jednotlivých polí ve zprávě a je reprezentován znakem |.
- **Identifikátor:** Klíčové slovo, které určuje typ operace nebo události (např. LOGIN).
- **Parametry:** Datová část zprávy obsahující konkrétní hodnotu. Tato část není vyžadována u všech zpráv.
- **Ukončovací znak:** Každá zpráva je zakončena znakem nového řádku \n, který slouží k jednoznačnému určení konce zprávy.

3.1.2 Zprávy protokolu

Následující tabulka obsahuje přehled veškerých zpráv implementovaného protokolu.

Klient (Žádost)	Server (Odpověď)
LOGIN <přezdívka>	LOGIN <číslo_kódu>
PING	PONG <stav_klienta>
DISCONNECT	— (Ukončení spojení)
FIND	WAITING <číslo_kódu> GAME START_ <hráčův_symbol> <protihráč> <pole>
MOVE <X> <Y>	TURN <číslo_kódu> <pole> <symbol_na_tahu> RESULT <číslo_kódu> <vítěz> (vítěz předán jen pokud WIN kód)
REMATCH	GAME START_ <hráčův_symbol> <protihráč> <pole> GAME REMATCH_WAIT
LEAVE	SYNC LOBBY (Potvrzení odchodu)
SYNC	SYNC <stav_hráče> (pokud WAITING nebo LOBBY) SYNC GAME <hráčův_symbol> <pole> <symbol_na_tahu> <protihráč> SYNC RESULT <protihráč> <pole> <vítěz>
Asynchronní zprávy (Iniciuje server bez přímé žádosti)	
—	GAME PAUSED (Soupeř se odpojil)
—	GAME RESUMED <symbol_na_tahu> (Soupeř se vrátil)
—	GAME ENDED (Zápas byl zrušen)
—	AUTH 1 (vyzvání k autorizaci)

Tabulka 1: Přehled zpráv komunikačního protokolu

3.1.3 Detailní popis zpráv protokolu

V této sekci je definován význam všech zpráv, jejich směr (Klient → Server nebo naopak) a stavy, ve kterých jsou legitimní.

Autorizace a přihlášení

- AUTH|<1> (Server → Klient)
 - **Popis:** Úvodní výzva k autorizaci odeslaná serverem ihned po navázání spojení.
 - **Stav klienta:** LOGIN.
- LOGIN|<přezdívka> (Klient → Server)
 - **Popis:** Žádost uživatele o přihlášení do systému.
 - **Stav klienta:** LOGIN.
 - **Parametry:** přezdívka (povolená délka 4 až 12 znaků).
- LOGIN|<kód> (Server → Klient)
 - **Popis:** Odezva serveru na pokus o přihlášení.
 - **Návratové kódy:** 0: Úspěch; 1: Úspěšná obnova spojení (reconnect); 2: Server je plný; 3: Přezdívka je již obsazena; 4: Příliš dlouhá přezdívka; 5: Příliš krátká přezdívka.

Lobby a vyhledávání hry

- FIND (Klient → Server)
 - **Popis:** Žádost o vyhledání volného soupeře nebo zařazení do čekací fronty
 - **Stav klienta:** LOBBY nebo WAITING
- WAITING|<kód> (Server → Klient)
 - **Popis:** Informace o stavu zařazení do čekací místnosti.
 - **Návratové kódy:** 0: Uživatel byl úspěšně zařazen do fronty; 1: Všechny herní místnosti jsou obsazeny.
- GAME|START_|<symbol>|<protihráč>|<pole> (Server → Klient)
 - **Popis:** Oznámení o nalezení soupeře a zahájení nové hry.
 - **Stav klienta:** Přejít do IN_GAME.
 - **Parametry:** symbol: Přidělený symbol (X nebo O); protihráč: Přezdívka soupeře; pole: Aktuální stav pole.

Herní logika

- **MOVE|<X>|<Y>** (Klient → Server)
 - **Popis:** Požadavek na umístění symbolu na zadané souřadnice.
 - **Stav klienta:** IN_GAME.
 - **Parametry:** X, Y: Souřadnice v poli (rozsahu od 0 do 2).
- **TURN|<kód>|<pole>|<symbol_na_tahu>** (Server → Klient)
 - **Popis:** Potvrzení o zpracování tahu a aktualizace stavu hracího pole.
 - **Návratové kódy:** 0: Platný tah; 1: Hráč není na tahu; 2: Nevalidní souřadnice; 3: Pole je již obsazeno; 4: Hráč nepatří do této místnosti; 5: Hra aktuálně neběží.
 - **Parametry:** pole: Nový stav hracího pole; symbol_na_tahu: Kdo je aktuálně na řadě.
- **RESULT|<kód>|<vítěz>** (Server → Klient)
 - **Popis:** Oznámení o ukončení hry z důvodu vítězství nebo remízy.
 - **Stav klienta:** Přejít do RESULT.
 - **Návratové kódy:** 0: Hra skončila výhrou jednoho z hráčů; 1: Hra skončila remízou.
 - **Parametry:** vítěz: Přezdívka vítěze (u remízy prázdné).
- **REMATCH** (Klient → Server)
 - **Popis:** Žádost o zahájení odvetu se stejným protihráčem po ukončení zápasu.
 - **Stav klienta:** RESULT.
- **GAME|REMATCH_WAIT** (Server → Klient)
 - **Popis:** Informuje klienta, že jeho žádost o odvetu byla serverem přijata a nyní se čeká na vyjádření protihráče.
 - **Stav klienta:** RESULT.

Synchronizace a výpadky

- SYNC (Klient → Server)
 - **Popis:** Žádost o zaslání aktuálních dat pro obnovu stavu aplikace po znovupřipojení.
 - **Stav klienta:** SYNCING.
- SYNC|<stav>|... (Server → Klient)
 - **Popis:** Kompletní balík dat pro synchronizaci klienta se serverem.
 - **Parametry:** Struktura se liší dle stavu (např. pro **GAME** obsahuje: symbol, pole, tah, soupeř).
- GAME|PAUSED/RESUMED (Server → Klient)
 - **Popis:** Informuje o odpojení (PAUSED) nebo návratu (RESUMED) soupeře do probíhající hry.
- LEAVE (Klient → Server)
 - **Popis:** Žádost o okamžité opuštění aktuální herní místnosti nebo čekací fronty a návrat uživatele zpět do lobby.
 - **Stav klienta:** WAITING, IN_GAME nebo RESULT.
- DISCONNECT (Klient → Server)
 - **Popis:** Žádost o korektní ukončení spojení se serverem a odhlášení uživatele z aplikace.
 - **Stav klienta:** LOBBY.

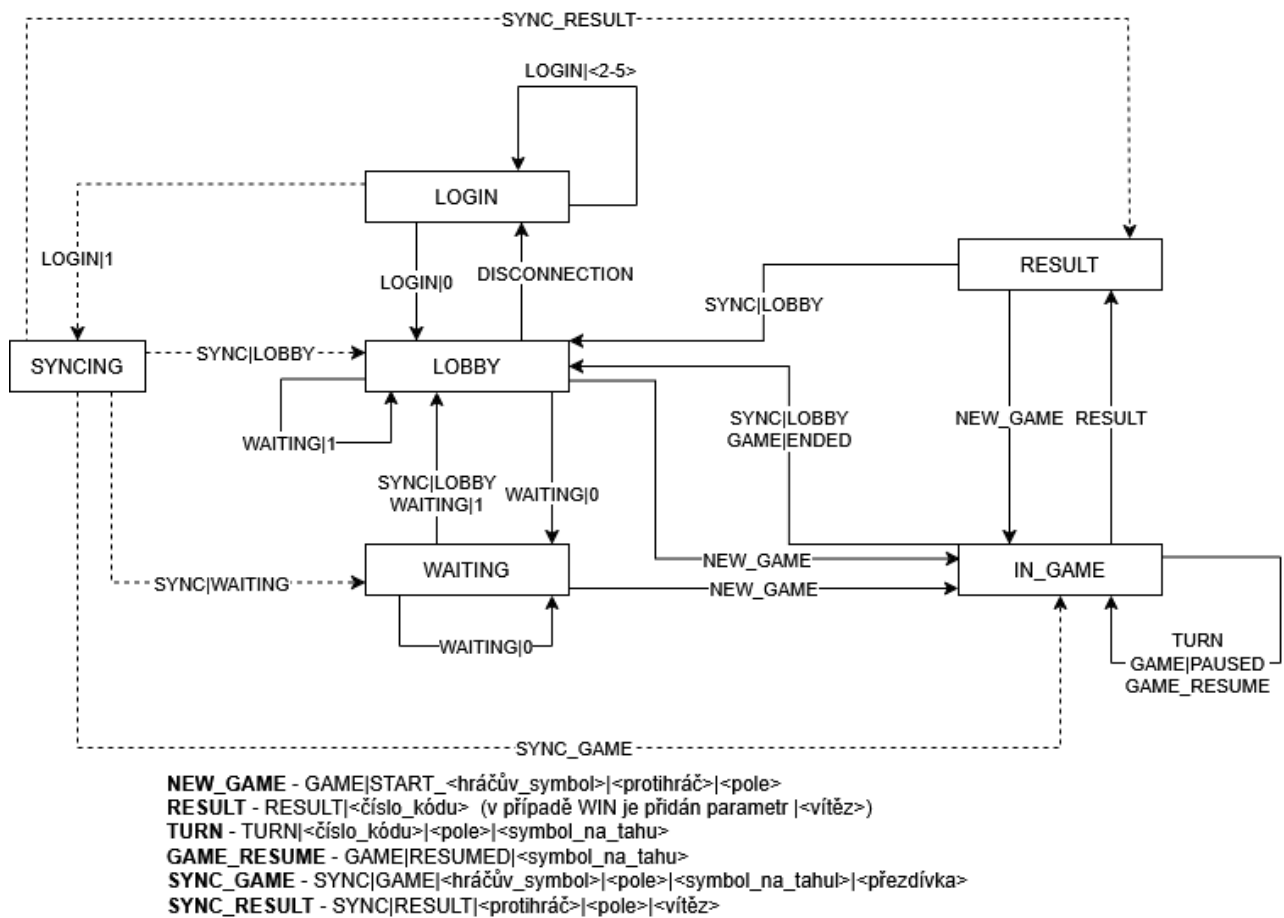
3.2 Popis stavů klienta

Klientská aplikace je implementována jako stavový automat, kde každý stav odpovídá konkrétnímu rozhraní v GUI a sadě povolených operací. Přejít mezi těmito stavy je řízen výhradně zprávami od serveru nebo explicitními akcemi uživatele jako je odpojení od serveru.

3.2.1 Seznam stavů

- **LOGIN:** Výchozí stav aplikace po spuštění. Klient v tomto stavu čeká na zadání přihlašovacích údajů uživatelem a následně navázání síťového spojení.
- **SYNCING:** Přejídný stav, který je aktivován po úspěšném znovupřipojení k serveru (reconnect), indikovaný kódem `LOGIN|1`. V tomto stavu klient automaticky odesílá zprávu `SYNC` a očekává od serveru obdržení nezbytných dat pro obnovu herního rozhraní do stavu před výpadkem.
- **LOBBY:** Hlavní rozcestník aplikace, do kterého se uživatel dostane po úspěšném prvotním přihlášení s kódem `LOGIN|0`. Uživatel zde má možnost požádat o vyhledání nové herní místnosti pomocí zprávy `FIND` nebo se od serveru validně odpojit zprávou `DISCONNECT`.
- **WAITING:** Stav, ve kterém je hráč zařazen do čekací fronty a vyčkává na připojení druhého soupeře po obdržení zprávy `WAITING|0`. Z tohoto stavu lze buď přejít přímo do zahájené hry (`NEW_GAME`), nebo se na žádost uživatele vrátit zpět do lobby odesláním zprávy `LEAVE`.
- **IN_GAME:** Stav představující aktivní hru. Klient zpracovává a vizualizuje tahy hráčů (`TURN`), reaguje na případné pozastavení hry při výpadku soupeře (`GAME|PAUSED`) nebo na její opětovné obnovení (`GAME|RESUME`).
- **RESULT:** Stav po ukončení zápasu a přijetí zprávy `RESULT`, ve kterém je uživateli zobrazen konečný výsledek hry. Hráč má v tomto bodě možnost požádat o okamžitou odvetu zprávou `REMATCH` nebo se vrátit do lobby žádostí přes signál `LEAVE`.

3.2.2 Diagram stavů klienta



Obrázek 1: Stavový diagram klienta a přechody mezi stavy

3.3 Implementace serveru

Server je implementován v jazyce C++ s využitím nízkoúrovňových BSD socketů. Pro zajištění stability a přehlednosti je kód rozdělen do několika logických celků:

- **Server:** Hlavní třída spravující životní cyklus serveru, inicializaci socketů a hlavní síťovou smyčku.
- **UserManager:** Modul obsahující funkce zodpovědné za správu připojených uživatelů, jejich autentizaci a sledování aktivity pro detekci timeoutů.
- **RoomManager:** Správce herních místností, který zajišťuje párování hráčů do her a izolaci jednotlivých zápasů.
- **Room:** Třída implementující samotnou herní logiku piškvorek 3x3, kontrolu vítězných podmínek a správu tahů.

Pro paralelní obsluhu více klientů a herních místností funkci `select()`. Tato metoda umožňuje serveru v jediném vlákně neblokujícím způsobem monitorovat aktivitu na všech připojených deskriptorech najednou.

3.4 Implementace klienta

Klient je implementován v jazyce Python s využitím frameworku PyQt6, který zajišťuje grafické uživatelské rozhraní.

- **NetworkClient:** Tato třída běží v samostatném vlákně, aby síťová komunikace neblokovala odezvu uživatelského rozhraní. Zajišťuje nízkoúrovňové čtení ze socketu, skládání zpráv a jejich parsování. Implementace této třídy je k nalezení v souboru `sockets.py`
- **scene_manager:** Soubor zajišťující přepínání mezi jednotlivými scénami (Login, Lobby, Game) na základě signálů z `NetworkClient`.

4 Uživatelská dokumentace

Tato sekce popisuje postupy pro kompilaci a spuštění obou částí aplikace a základní principy ovládání grafického rozhraní klienta.

4.1 Obsluha serveru

Server je vyvíjen pro prostředí GNU/Linux a k jeho sestavení se využívá nástroj `make`.

4.1.1 Kompilace serveru

Pro úspěšný překlad je vyžadován kompilátor `g++` s podporou standardu `C++17`. Sestavení se provádí příkazem: `make`. Tento proces vytvoří složku `obj/` pro objektové soubory a složku `bin/`, která obsahuje výsledný spustitelný soubor `server.exe`.

4.1.2 Spuštění serveru

Server se spouští z terminálu s následujícími povinnými parametry, které definují síťové nastavení a limity zdrojů:

```
./server.exe <-p <PORT> -c <MAX_CLIENT> -r <MAX_ROOMS> [-a <IP>]
```

Parametry umožňují konfigurovat naslouchající port a celkovou kapacitu serveru bez nutnosti zásahu do zdrojového kódu.

Popis parametrů pro spuštění

- `-p <PORT>`: Číslo portu, na kterém bude server naslouchat. Podporovaný rozsah je 1024 až 65535.
- `-c <MAX_CLIENT>`: Maximální celkový počet klientů. Minimální hodnota jsou 2 hráči, aby bylo možné sestavit hru.
- `-r <MAX_ROOMS>`: Maximální počet herních místností. Minimální počet je 1 místnost.
- `-a <IP>`: (Volitelné) IP adresa, na které bude server naslouchat. Pokud není tento parametr zadán, server defaultně naslouchá na všech dostupných rozhraních (0.0.0.0).

4.2 Obsluha klienta

Klientská aplikace je vytvořena v jazyce Python s využitím grafického frameworku PyQt6. Celý proces přípravy prostředí a následného spuštění je automatizován pomocí pomocného skriptu `run_client.py`

4.2.1 Příprava prostředí

Před samotným spuštěním aplikace na operačním systému Linux je nutné zajistit přítomnost systémových knihoven nezbytných pro běh grafického rozhraní v prostředí:

```
sudo apt update && sudo apt install -y libxcb-cursor0 libxcb-xinerama0  
libxcb-icccm4 libxcb-image0 libxcb-keysyms1 libxcb-render-util0 libxcb  
-shape0
```

Správa závislostí

Skript `run_client.py` při svém spuštění automaticky vytvoří izolované virtuální prostředí ve složce `.venv` a nainstaluje veškeré potřebné knihovny definované v souboru `pack_requirements.txt`.

4.2.2 Spuštění klienta

Vlastní spuštění aplikace se provádí skrze terminál voláním zmíněného souboru `run_client.py`, který nastaví vše potřebné a spustí hlavní modul aplikace.

- **Windows:** `python run_client.py`
- **Linux:** `python3 ./run_client.py`

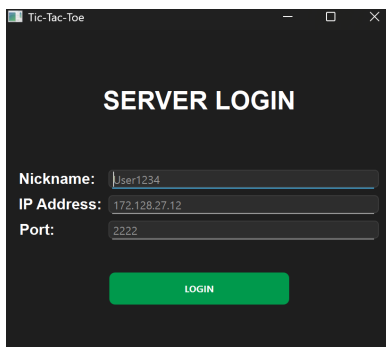
Tento postup zajišťuje, že aplikace běží vždy v korektně nastaveném prostředí se všemi vyžadovanými verzemi knihoven bez nutnosti ruční konfigurace uživatelem.

4.3 Základní ovládání klienta

Uživatelské rozhraní klientské aplikace je navrženo jako intuitivní grafické prostředí. Ovládání je rozděleno do několika logických scén, které vedou uživatele celým herním procesem.

4.3.1 Přihlášení do systému (LoginScene)

Po spuštění aplikace je uživatel vyzván k zadání konfiguračních údajů pro spojení se serverem.



Obrázek 2: Přihlašovací scéna aplikace

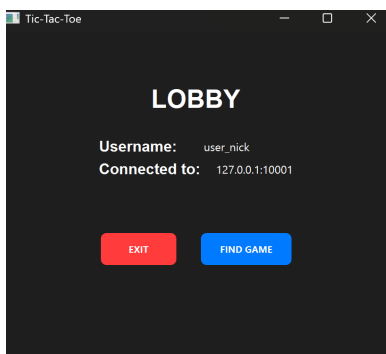
Uživatel musí vyplnit následující pole:

- **Nickname:** Přezdívka hráče o délce 4 až 12 znaků.
- **IP Address:** Síťová adresa běžícího serveru.
- **Port:** Číslo portu v rozsahu 1024 až 65535.

Pokud jsou údaje v pořádku, kliknutím na tlačítko **LOGIN** dojde k pokusu o navázání spojení a odeslání autorizační zprávy.

4.3.2 Hlavní lobby (LobbyScene)

Lobby scéna představuje centrální rozcestník aplikace.



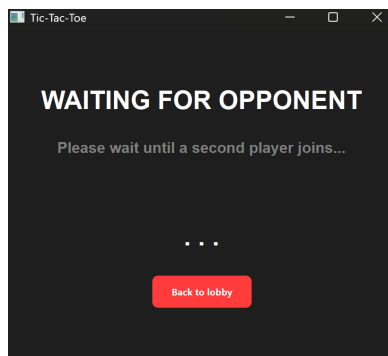
Obrázek 3: Uživatelské rozhraní lobby

V horní části lobby jsou zobrazeny informace o přihlášeném uživateli a adrese serveru. Hráč má k dispozici dvě hlavní akce:

- **FIND GAME:** Odešle serveru žádost o vyhledání volného soupeře.
- **EXIT:** Proveďte korektní odhlášení, uzavření socketu a návrat na přihlašovací obrazovku.

4.3.3 Čekací místnost (WaitingScene)

Pokud server nemá okamžitě k dispozici volného protihráče, je klient zařazen do fronty a zobrazí se scéna čekání.

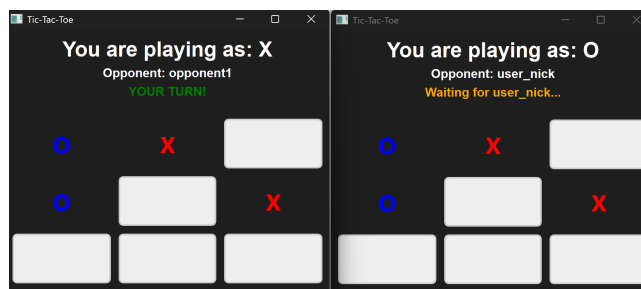


Obrázek 4: Obrazovka čekání na protihráče

Tato scéna obsahuje animovaný indikátor průběhu potvrzující aktivitu aplikace. Hráč může kdykoliv kliknout na tlačítko **Back to lobby**, čímž zruší svou žádost o hru a server jej vyřadí z čekací fronty.

4.3.4 Průběh herního zápasu (GameScene)

Jakmile server spáruje dva hráče, inicializuje se herní plocha o velikosti 3x3.



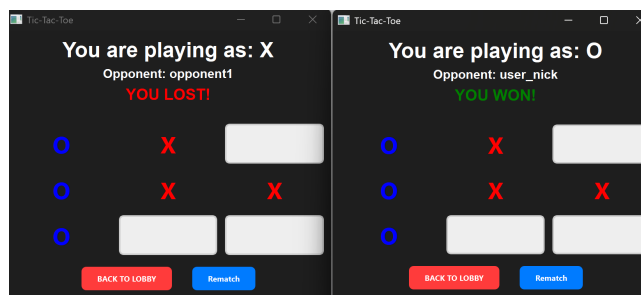
Obrázek 5: Scéna probíhajícího zápasu z pohledu obou hráčů

Ovládací prvky a indikátory hry zahrnují:

- **Herní pole:** Kliknutím na volné tlačítko hráč provede svůj tah.
- **Stavový řádek:** Řádek informující hráče o aktuálním tahu (**YOUR TURN!** nebo **Waiting for opponent...**).
- **Synchronizace:** Pokud se protihráč odpojí, pole se zablokuje a zobrazí se hlášení **GAME PAUSED**. Po znovupřipojení soupeře se stav automaticky obnoví.

4.3.5 Výsledek hry a odvěta

Po dosažení konce hry se zobrazí informace týkající se výsledku hry.



Obrázek 6: Zobrazení výsledku hry a možnosti dalšího postupu

V tomto stavu má uživatel dvě možnosti:

- **REMATCH:** Žádost o novou hru se stejným protihráčem. Ta je zahájena pouze v případě, že o odvetu požádají oba účastníci.
- **BACK TO LOBBY:** Ukončení herní relace v dané místnosti a návrat hráče do lobby pro vyhledání nového soupeře.

5 Závěr

V rámci této semestrální práce jsem úspěšně navrhl a naimplementoval hru Piškvorky, která využívá architekturu klient-server a vlastní textový protokol postavený nad transportním protokolem TCP. Výsledný program splňuje veškeré požadavky stanovené zadáním, včetně paralelní obsluhy více hráčů a schopnosti zotavení po nečekaném výpadku spojení.

Tento projekt byl mojí první větší zkušeností s programováním v jazyce C++. Práce na něm mě velmi bavila a pomohla mi lépe pochopit nízkoúrovňovou komunikaci, správu paměti a návrh rozsáhlejších aplikací s grafickým rozhraním v PyQt6. Získal jsem díky němu cenné zkušenosti, které určitě využiji v dalším studiu i budoucí praxi.

Program je plně funkční a připravený pro potenciální budoucí rozšíření. Mezi tato vylepšení by mohla patřit například implementace žebříčku nejlepších hráčů, podpora většího rozměru herního pole s dynamickou výherní podmínkou nebo přidání herního chatu pro komunikaci mezi protihráči.